27th October 2026

An Introduction to Tabular Foundation Models

by Valentin Kilian

I) Prior Fitted Network

[Transformers can do Bayesian inference , ICLR 2022
S. Müller , N. Hollmann, S. Pineda, J. Grabocka, F. Hutter]

Supervised ML → Big success

BUT You need a lot of training data

How to transfer these successes to setups with less data available ?

⟹ Encoding prior knowledge into the ML model so it learns faster

How to encode prior info in a ML model ?

- Model architecture (CNNs ...)
- Regulariser (Data augmentation)
- Bayesian Analysis

BA for prediction in supervised ML

Set up :

Input x Output y

Data $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$ (Labelled dataset)

Goal : Learn $\mathbb{P}(y \mid x, \mathcal{D})$

Idea

- y latent variable
- Prior $p(y)$
- Posterior $p(y \mid D)$
- Posterior predictive

$$p(y \mid x, D) = \int p(y \mid x, y)\, p(y \mid D)$$

✓ Good theoretical foundations

✓ Better account for the actual likelihood of different events

✓ It well calibrated

✓ Interpretable

✗ Retrieving the posterior predictive is intractable in most case

Existing methods to approximate the predictive

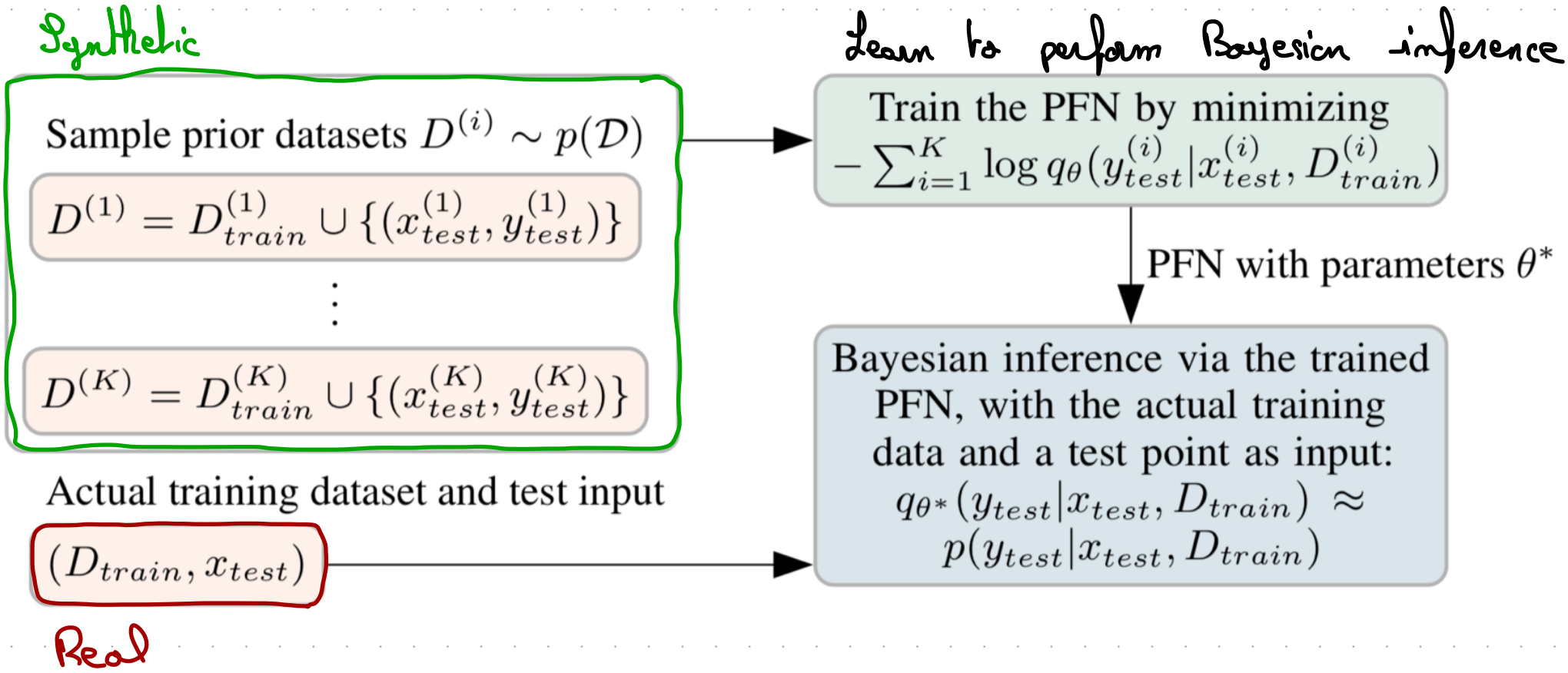
- MCMC, HMC, NUTS
- Variational Inference
- Simulation-based inference
- Neural Process

Here they introduce

Prior-Data Fitted Network

PFN can approximate the posterior predictive for any prior from which we are able to sample data.

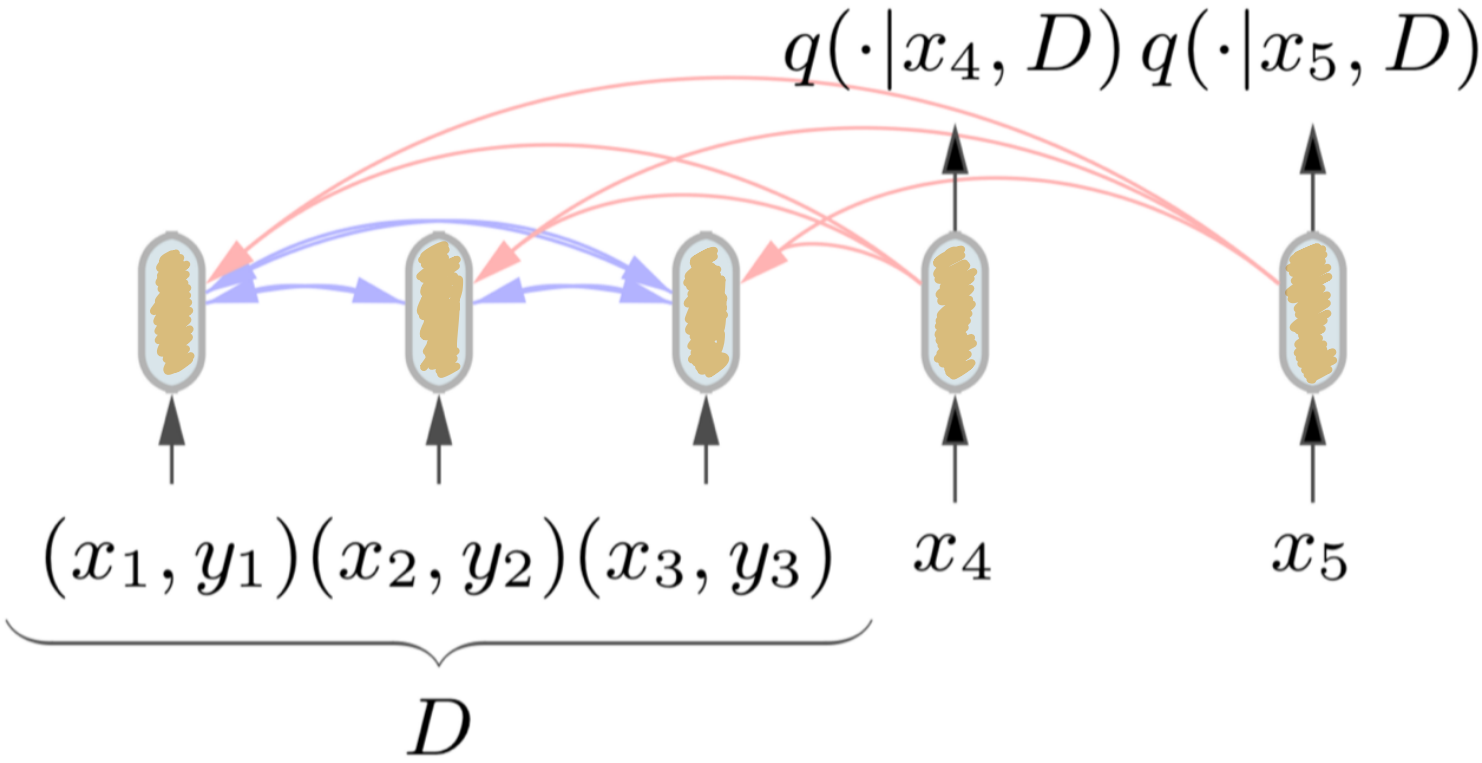
Prior - Data Fitted Network



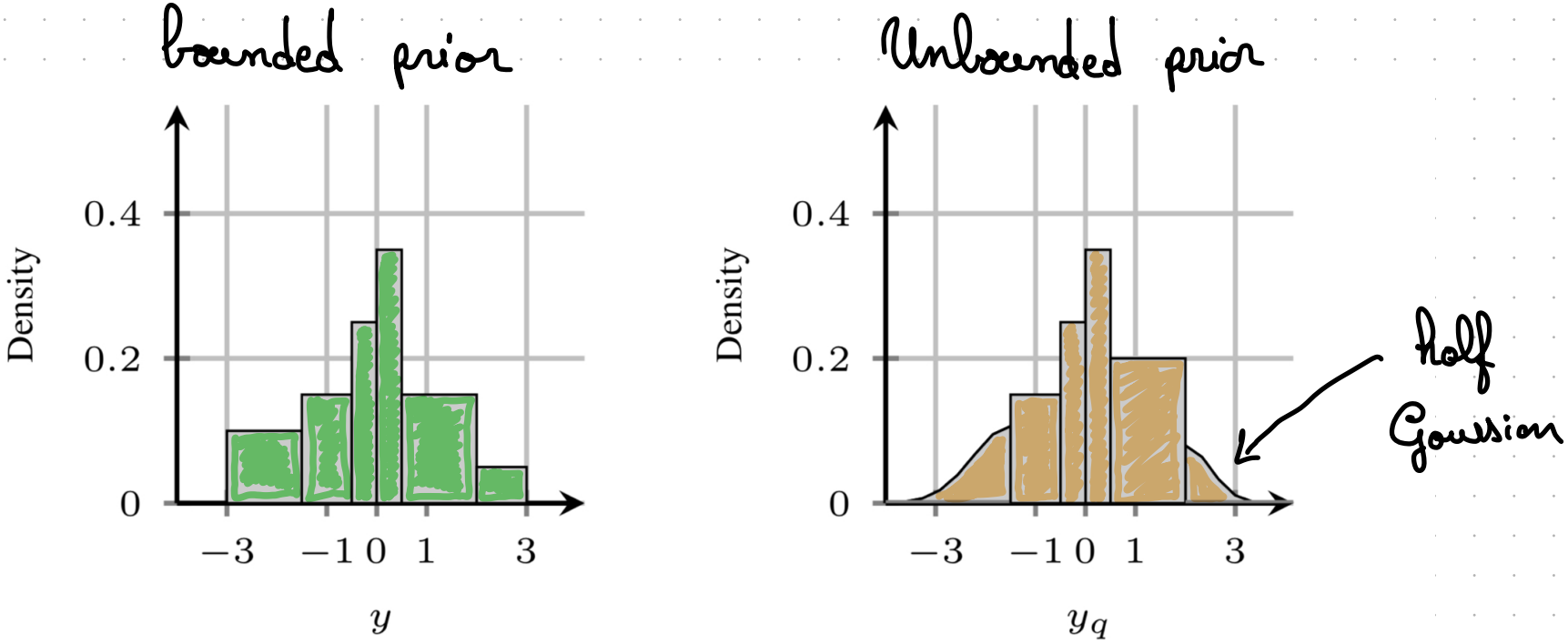
Adapting the Transformer for Bayesian Inference

Parametric approximation of the posterior predictive q_θ

Architecture : Transformer encoder without positional encoding.



Output : Riemann Distribution



- output could be an continuous distribut°
- turn it into a " classification" problem
- buckets boundaries are select such as each bucket have same proba under the prio

Training a PFN

Algorithm 1: Training a PFN model by Fitting Prior-Data

Input : A prior distribution over datasets $p(\mathcal{D})$, from which samples can be drawn and the number of samples K to draw

Output : A model q_θ that will approximate the PPD

Initialize the neural network q_θ;

for $j \leftarrow 1$ **to** K **do**

  Sample $D \cup \{(x_i, y_i)\}_{i=1}^{m} \sim p(\mathcal{D})$;

  Compute stochastic loss approximation $\bar{\ell}_\theta = \sum_{i=1}^{m}(-\log q_\theta(y_i|x_i, D))$;

  Update parameters θ with stochastic gradient descent on $\nabla_\theta \bar{\ell}_\theta$;

end

Loss : Cross - entropy over the sample drawn from the prior

$$\ell_\theta = \mathbb{E}_{D \cup \{x,y\} \sim p(D)} \left[-\log q_\theta(y \mid x, D) \right]$$

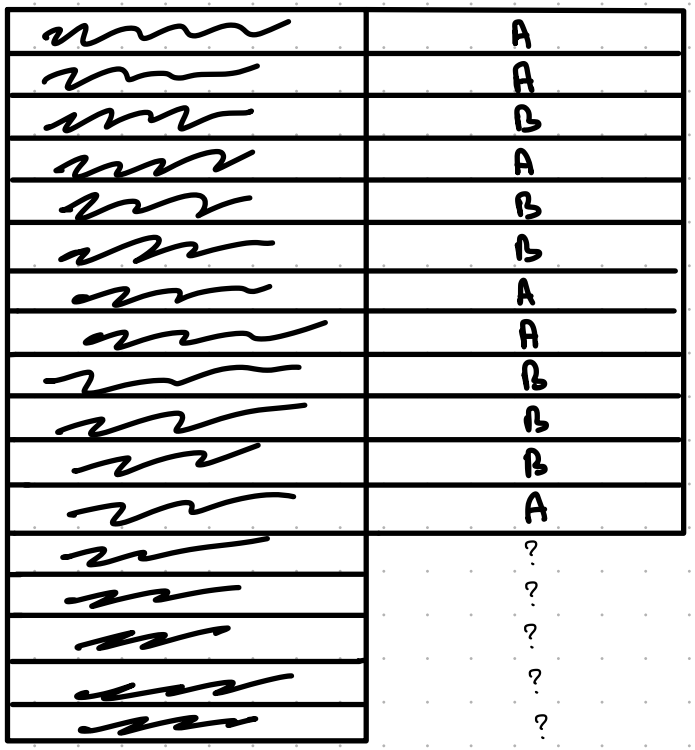
$$= \mathbb{E}_{x, D \sim p(D)} \left[H\left(p(\cdot \mid x, D), q_\theta(\cdot \mid x, D)\right) \right]$$

$$= \mathbb{E}_{x, D} \left[KL\left(p(\cdot \mid x, D), q_\theta(\cdot \mid x, D)\right) \right]$$

II) TabPFN

[TabPFN: A Transformer that Solves Small Tabular Classification Problem in a Second, ICLR 2023, N. Hollmann, S. Müller, K. Eggensperger, F. Hutter]

- Tabular data are the most common data type in real world ML
- BUT multiple small dataset



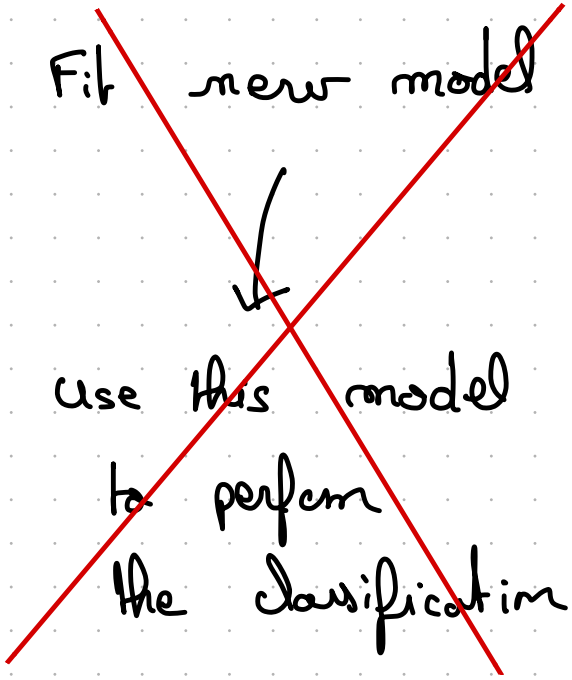
Training data

In most case, not enough data to train an ML model

Task

- Prior to TabPFN, tabular data classification was mostly done with Gradient-Boosted Decision Tree

Goal : using PFN to build a fondation model that would work for all tabular data



⇒

Zero shot classification via In Context Learning

⇒ We need a synthetic prior for tabular data

Prior for Tabular Data

- Priors base on a notion of "simplicity"
- Two families of prior:
  - ↳ Structural Causal Models (SCMs)
  - ↳ Bayesian Neural Networks (BNNs)

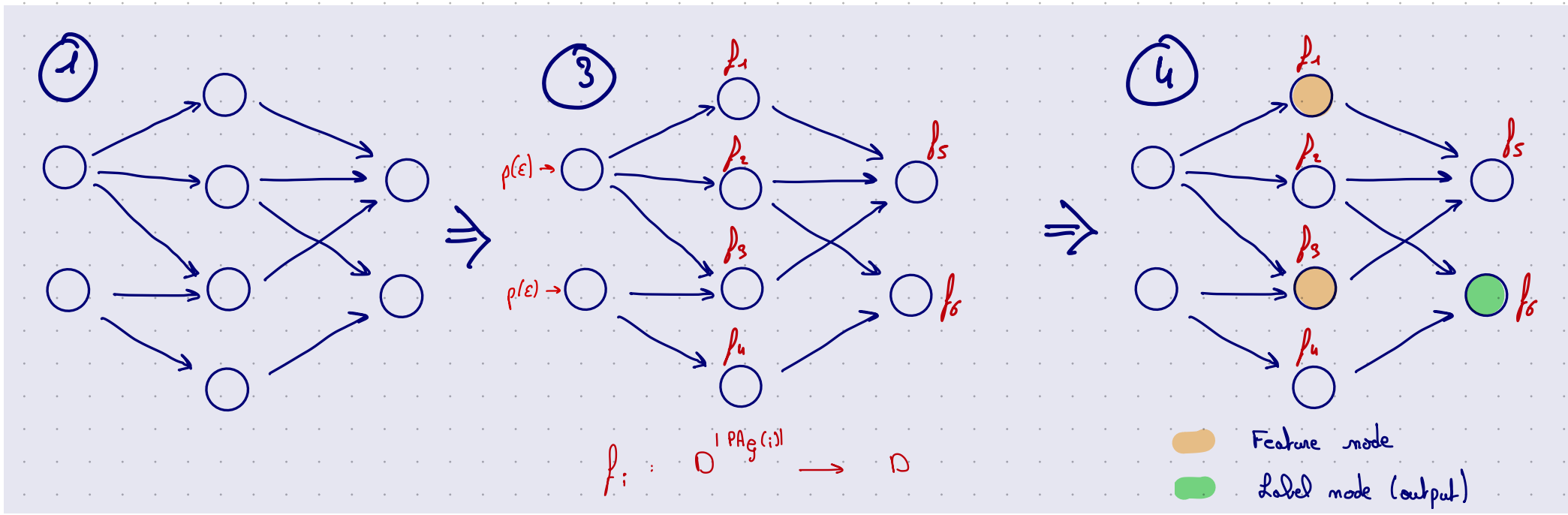
Sequential Causal Model

1) Sample one DAG G

2) Sample a noise distribution $p(\varepsilon) \sim p(p(\varepsilon))$
with support in a domain $\mathbb{D} \subset \mathbb{R}$

3) Sample 1 causal mechanism per non root nodes

4) Sample 6 features nodes and 1 label node



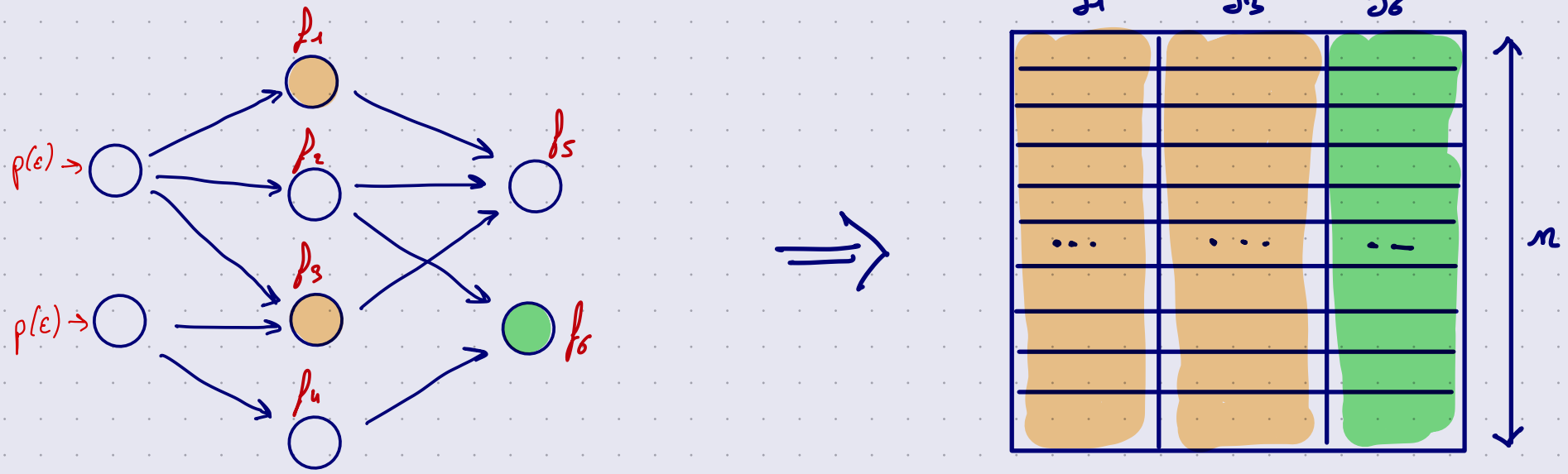
Sequential Causal Model

5) Sample a table with n rows by repeating n times

- Initialize each root node with $y_i \sim p(\epsilon)$
- Propagate in the DAG through

$$y_i = f_i(\{y_j, j \in PAG(i)\})$$

- Collect feature and label into a table



Bayesian Neural Network Prior

1) Sample a model architecture

$$A \sim p(A)$$

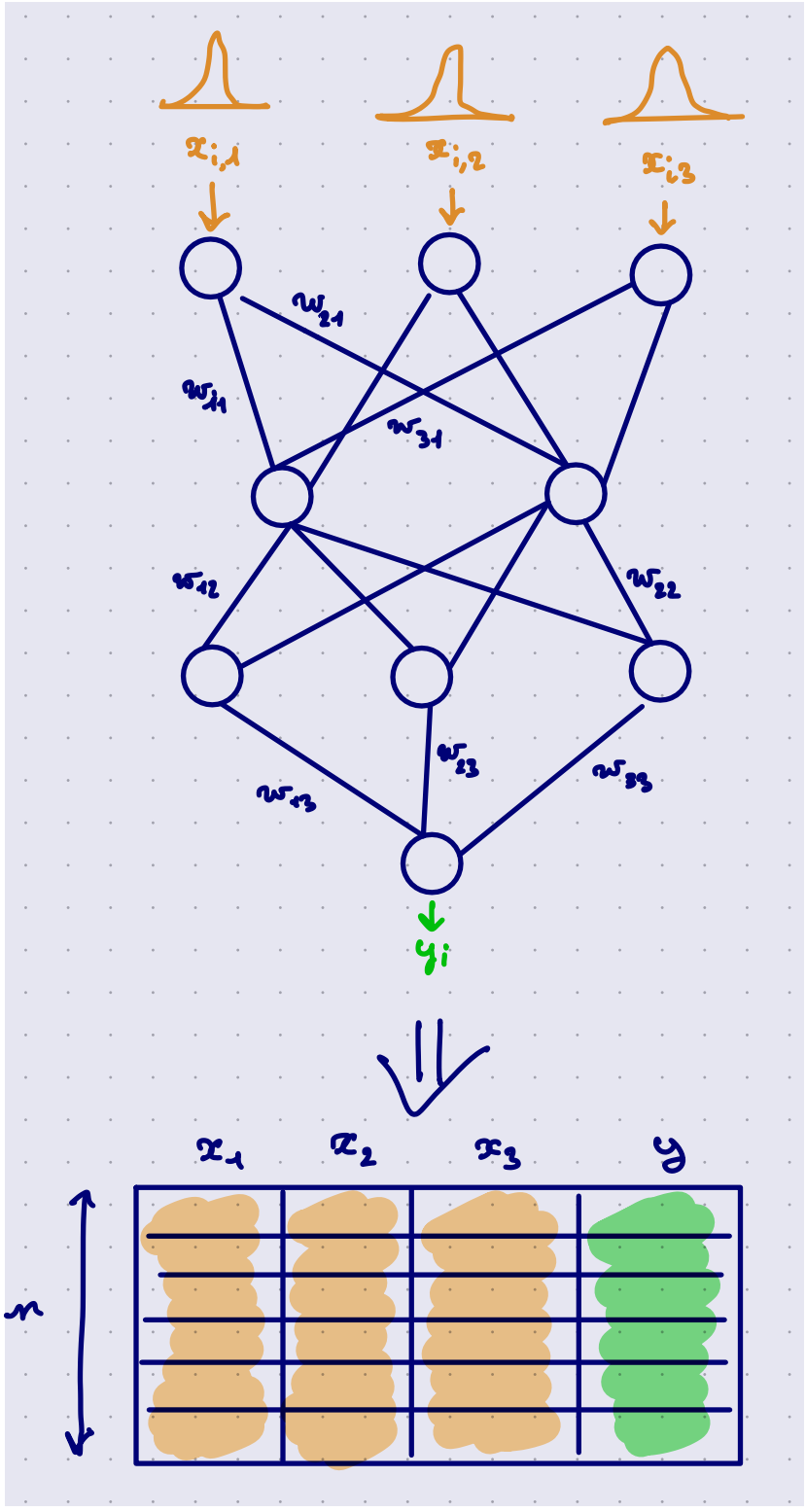
2) Sample weight for the architecture

$$w_{i,j} \sim p_w(\cdot)$$

3) Sample iid feature

$$x_{i,j} \sim \mathcal{N}(0,1)$$

4) Feed the NN with those features to get the output



Full prior construction

- Type $\sim \mathcal{U}(\{\text{SCM}, \text{BNN}\})$
- $(x_i^k, y) \sim \text{Type}$
- Sample number of class $N_c \sim p(N_c)$
  $p(N_c) \in \mathcal{P}(\mathbb{N})$
- Sample $N_c - 1$ class bounds randomly from the set of continuous target y
- Map each scalar y_i to its class

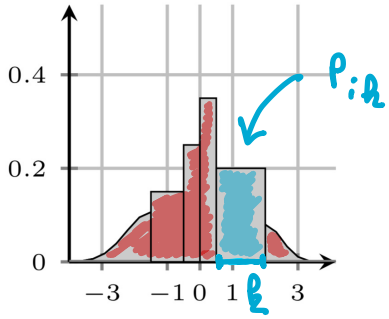
From scalar y to multi-class

Experiments

NB: The original papers contains an extensive experiment section, with ablation study, I only present a tiny subset

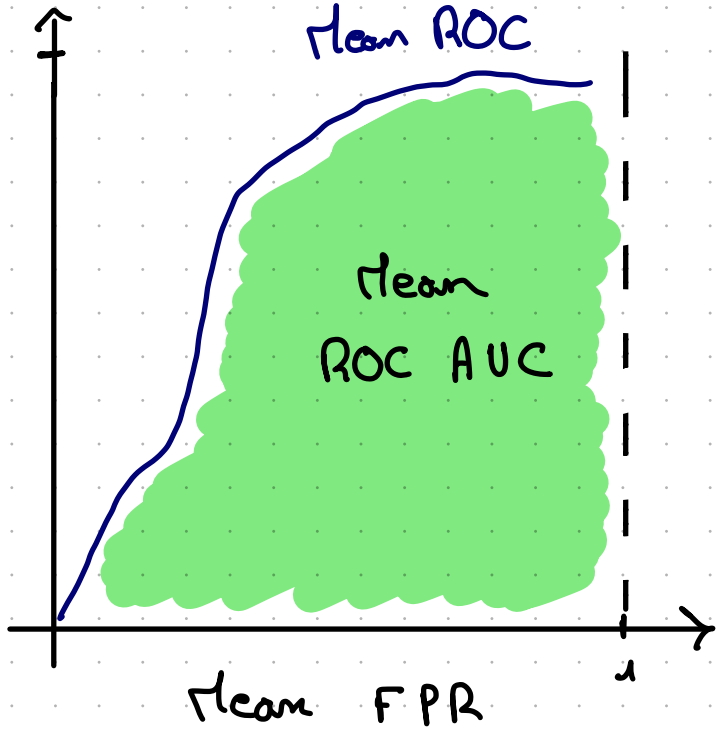
Reminder ROC AUC (Receiver operating Characteristic Area Under the Curve)

output



$$TPR_k(L) = \frac{\sum_{i, y_i = k} \mathbb{1}\{p_{i,k} \geq L\}}{\#\{i, y_i = k\}}$$

$$FPR_k(L) = \frac{\sum_{i, y_i \neq k} \mathbb{1}\{p_{i,k} \geq L\}}{\#\{i, y_i \neq k\}}$$



Experiments

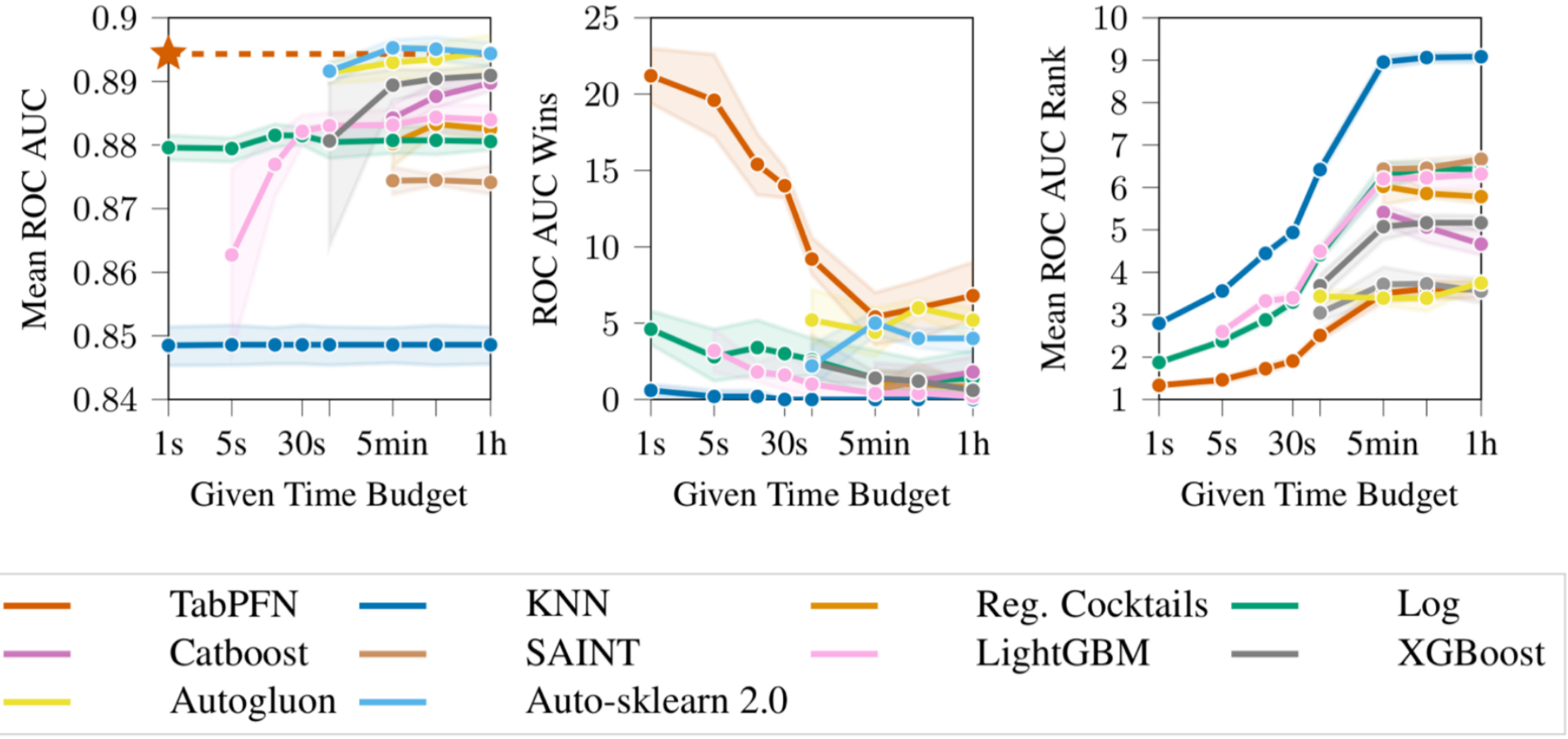


Figure 7: ROC AUC performance over time on the 30 small OpenML-CC18 including datasets with categorical features and missing values.

Limitations

- No real handling of missing values
- No real handling of categorical feature
- Only suitable for small dataset due to computational cost
- Only for classification not for regression

Subsequent work will focus on those issues by playing on two points:

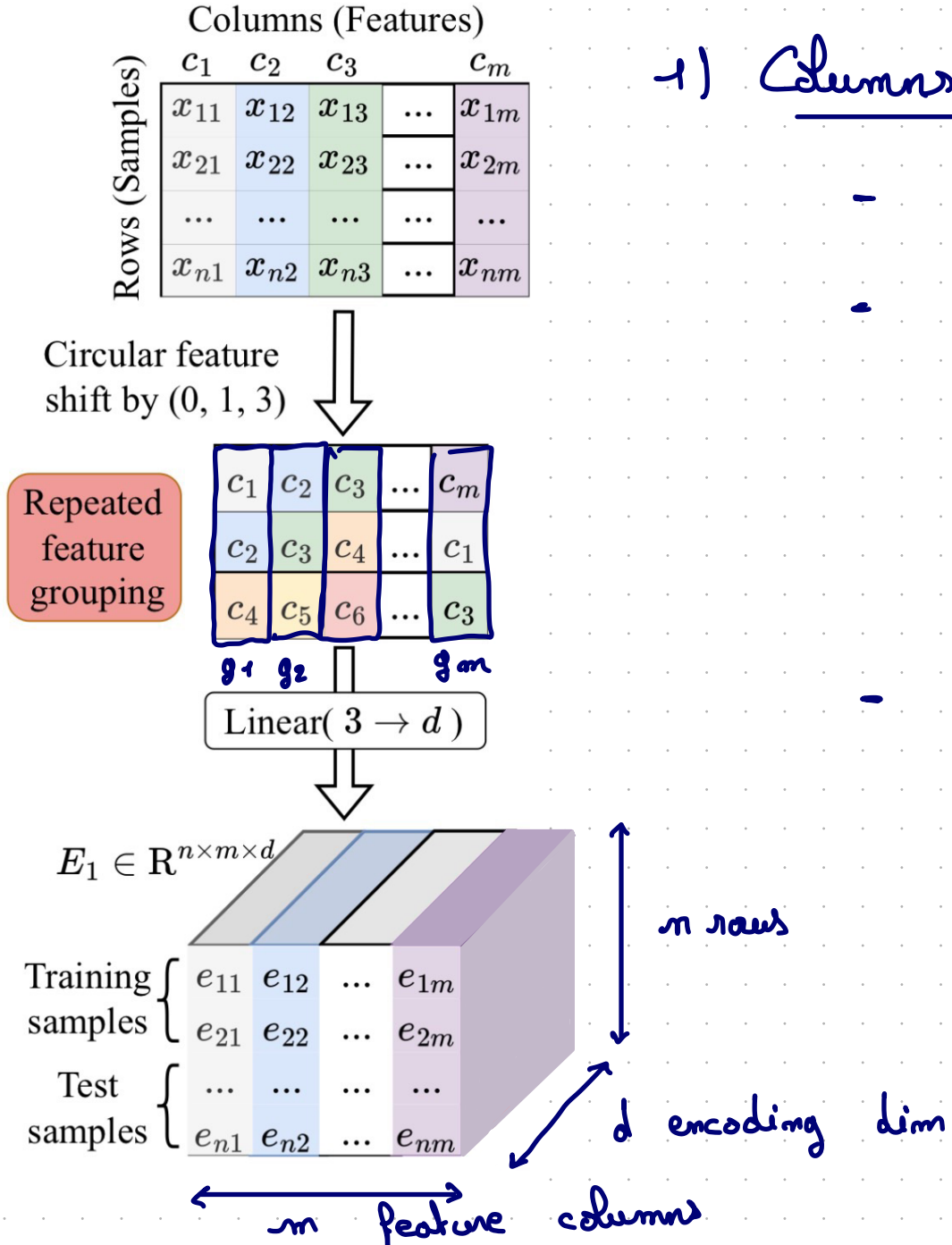
- Architecture
- Prior

III) TabICL v2 : Harder, Better, Faster, Stronger

[TabICLv2: A better, faster, scalable, and open tabular foundation model, ICML 2026, J. Qu*, D. Holzmüller*, G. Varoquaux, M. Le Morvan]

Model	Year	Venue	Max columns	Max rows
TabPFN	2023	ICLR	100	1000
TabPFN v2	2025	Nature	500	10 000
TabICL	2025	PMLR	2000	500 000
TabICLv2	2026	ICML	2000+	1 000 000

Architecture



1) Columns encoding · repeated feature grouping

- m columns
- create m groups : group g_j contains columns at position $(j, j+1, j+3) \bmod m$
- each group is encoded by a shared linear layer

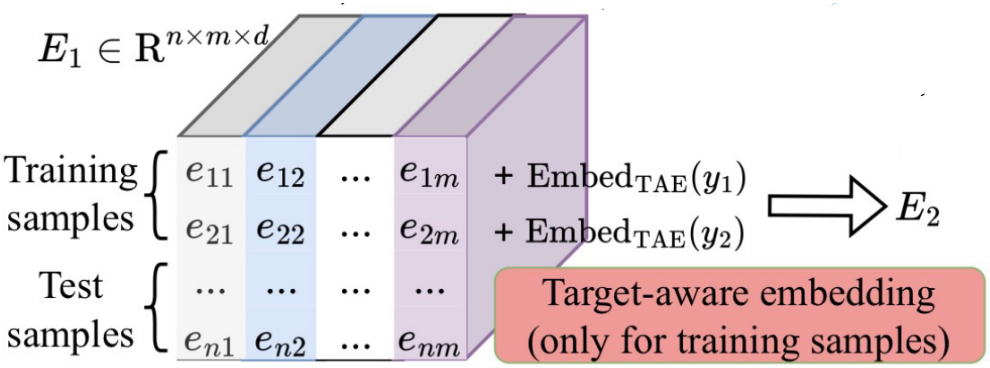
Architecture

2) Target aware embedding

Early injection of target information

$$E_2[i,j] = \begin{cases} E_1[i,j] + \text{Embed}_{TAE}(y_i) & \forall i \in \mathcal{D}_{train} \\ E_1[i,j] & \forall i \in \mathcal{D}_{test} \end{cases}$$

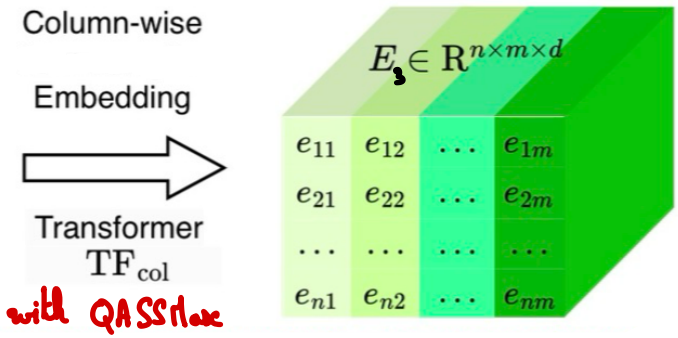
$$\text{Embed}_{TAE} : \text{Target space} \longrightarrow \mathbb{R}^{\phi}$$



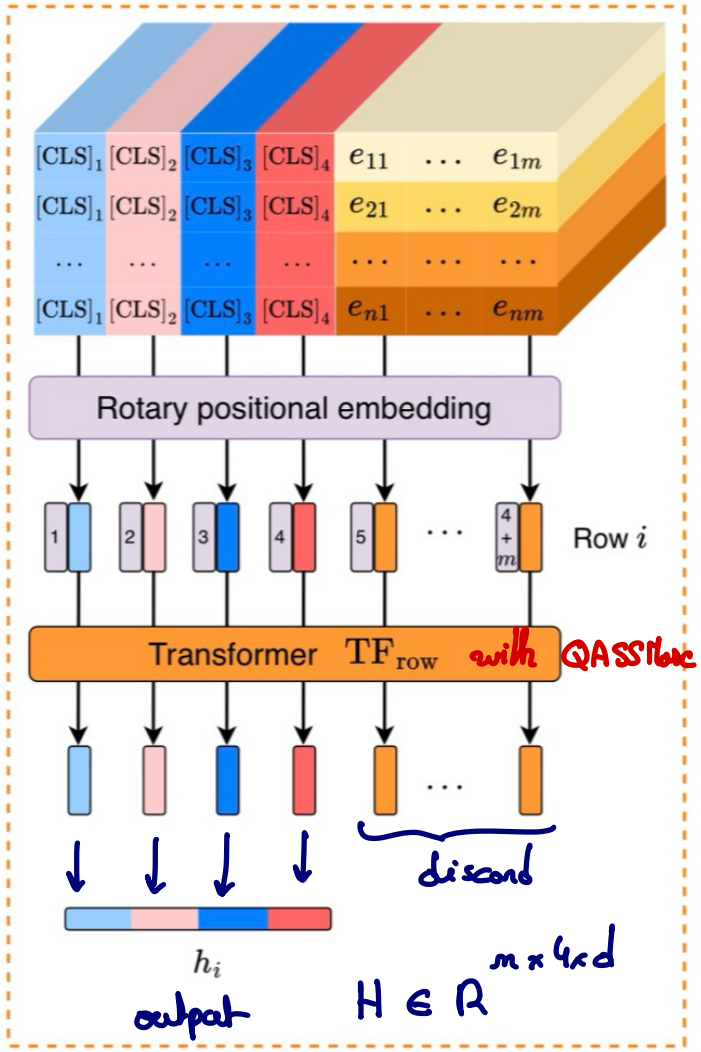
$$\in \mathbb{R}^{n \times m \times d}$$

Architecture

E_2



Row-wise inter-feature Interaction



3) Compression

(a) Column-wise : set of transformers TF_{col} apply column by columns on the tokens of dim d

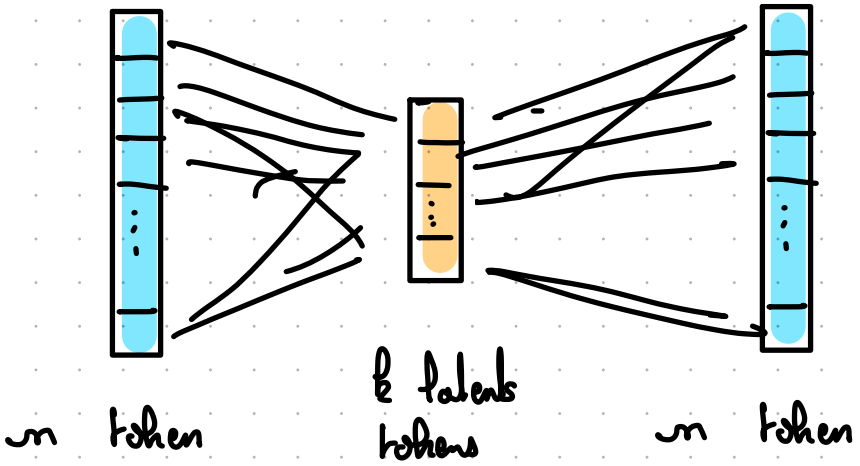
- induced self attention
- query-aware scalable softmax

(b) Row wise interaction:

1) Add 4 CLS learnable token for each row (same 4 token)
2) Add positional embedding for the columns position
3) Apply set of transformers T_{row} with full attention and QSSAMax
4) Keep only the 4 CLS output

Architecture

Induced self attention : kind of attention that is more computationally efficient



$O(n^2) \rightarrow O(kn)$

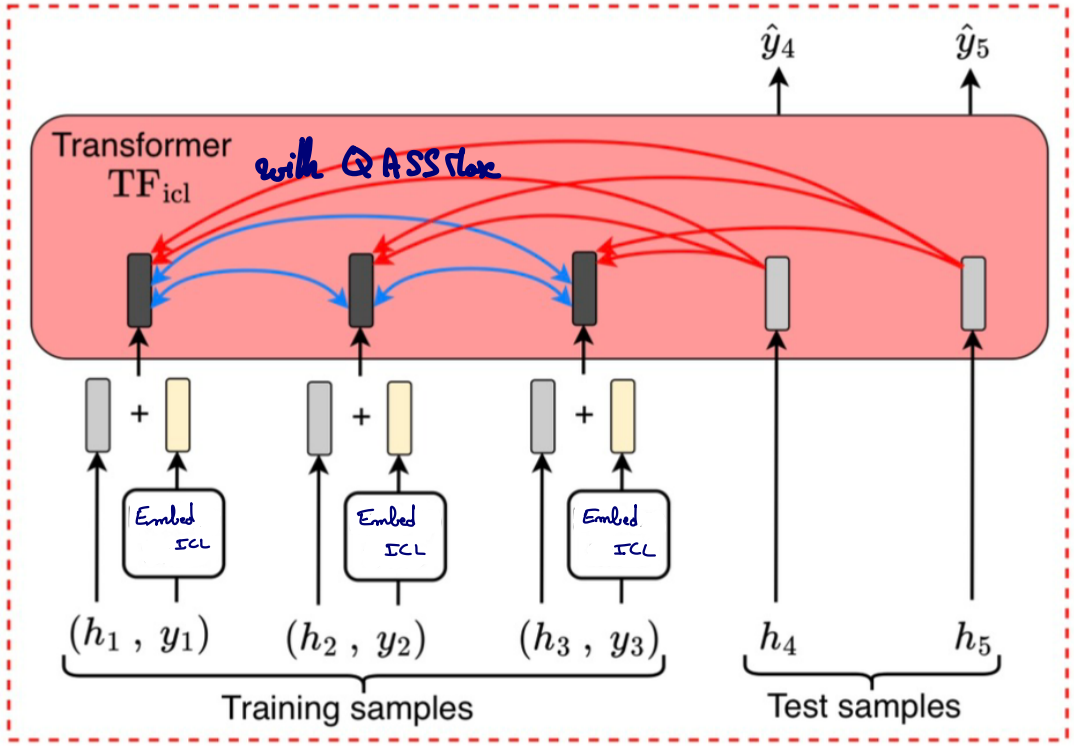
full self attention → induced self attention

Query - aware scalable softmax

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \quad \text{(regular self attention)}$$

QSSMax is a way to rescale the softmax to avoid attention fading when context is very long

Architecture



4) Dataset wise ICL

Same as in PFN

but with

- full attention
- QASSMax

5) Output : unlike TabPFN, TabICL v2 do not returned the discretized full predictive but 999 quantiles at probability level $\alpha \in \{0.001 \dots 0.999\}$, trained with pinball loss.

Optimizer

They use Nadam optimizer

Synthetic Prior

- Similar as in TabPFN but with improved generative process, in order to build more expressive and diverse prior. No more BNN prior.
- They also add a filter to remove synthetics dataset that contain little or no learnable relation between the feature and the output.

Experiments

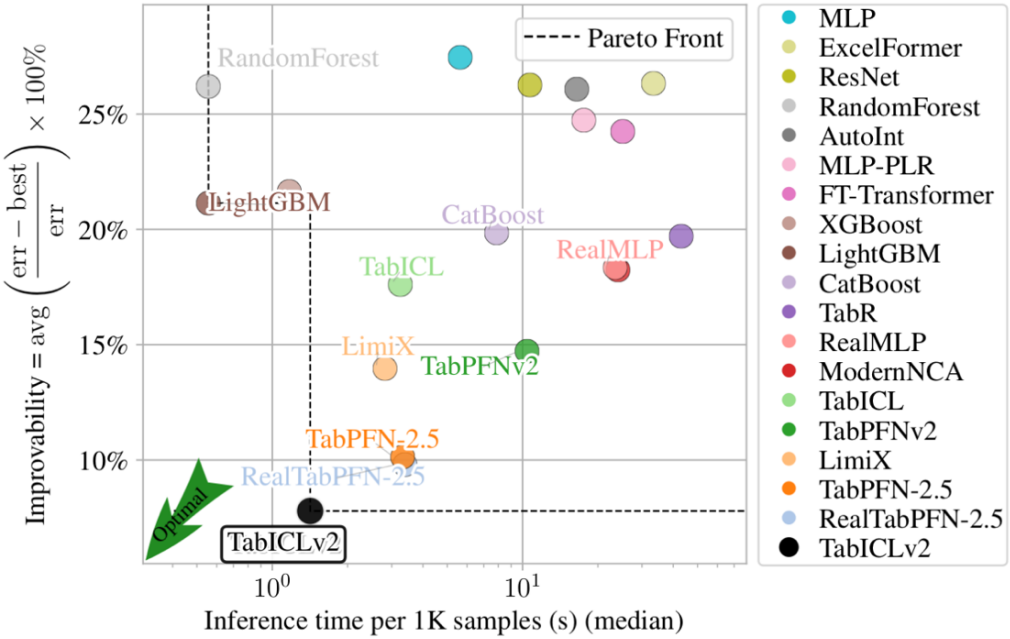


Figure 5. **Improvability vs. inference time on TALENT (Ye et al., 2024).** The runtime of TabICLv2 is measured on an H100 GPU, while other runtimes are taken from TALENT.

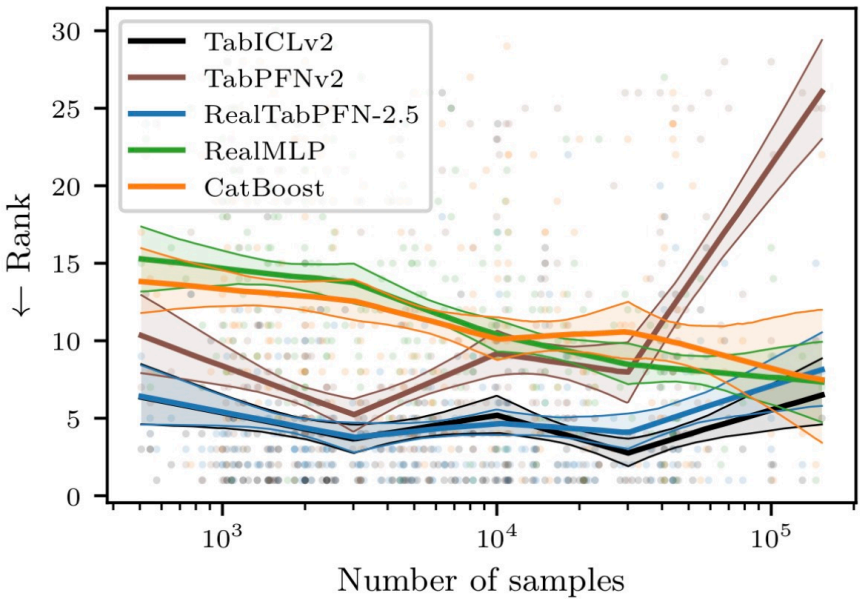


Figure 8. **Model rankings as a function of sample size on TALENT.** The lines show the bootstrap median and 10% / 90% bootstrap confidence intervals of a piecewise linear fit (Qu et al., 2025).

Conclusion : TabICL v2 is better and faster than existing model, it can also handle larger dataset

Experiments

Ref = TabICLv2 without QASSMax and per attention head.

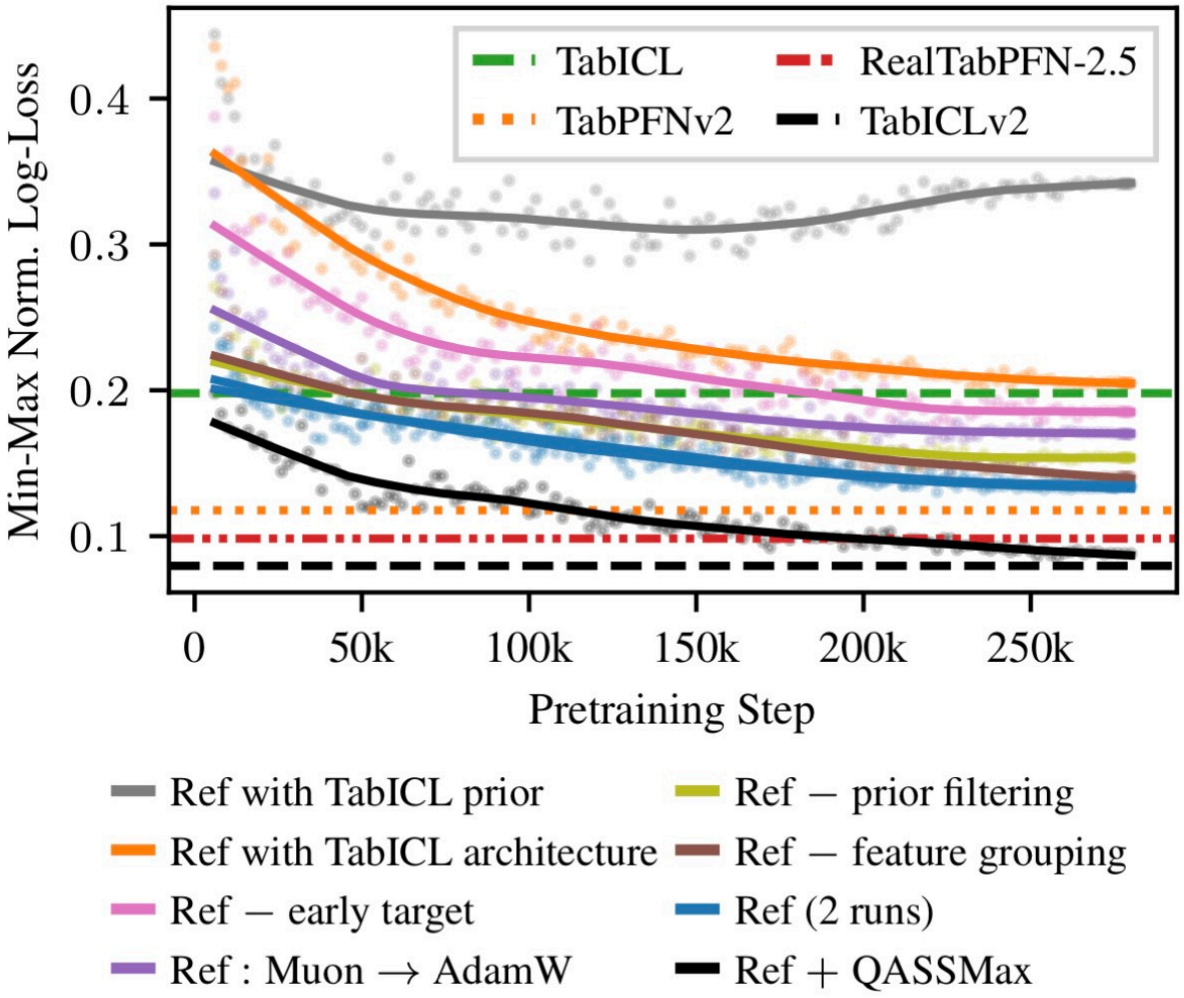


Figure 10. **Ablating different components of TabICLv2.** Non-

IV) Conclusion

- PFN is a ML paradigm where a transformers is trained on synthetic dataset to perform Bayesian inference
- PFN have allowed breakthrough in ML for tabular data leading to the first Tabular FM.
- It's becoming a trendy topic with new models regularly release (TabPFN-3.5, TabFM ...) and a lot of new start-up
- It has a lot of industry application including some in time series foundation model, causal inference...

V) Limitations

- Little theoretical work relative to its empirical succes
- Benchmarking those model is not solved yet (see Tab Arena & Beyond Tab Arena), the models works better on current benchmark than in the real world